LAB REPORT: CODE AND BRANCH COVERAGE ANALYSIS

Course: Software Engineering Lab (CS351) | Semester 5

Student Name	Shakthi Raja G P
SRN & Section	PES1UG25CS843 | Section L
Assigned Problem & Repository	Problem #60: Marathon Registration (SETAPESU26/L6_L7_60_marathon_registration)
Module Under Test	marathon_registration.py

1. Introduction and Objectives

Code coverage is an essential white-box testing technique that quantifies the degree to which source code is executed during automated test suites. This laboratory assignment investigates the distinction between Line (Statement) Coverage and Branch Coverage using the coverage measurement library on the assigned Marathon Registration system (Problem Statement 60).

Objectives: Key objectives of this laboratory exercise include:

1. Implementing CodeCoverageTest_marathon_registration.py to achieve >= 90% (achieved 100%) statement coverage.
2. Implementing BranchCoverageTest_marathon_registration.py to achieve >= 90% (achieved 100%) branch coverage across all control flow decision points.
3. Capturing real Alacritty terminal outputs showing test execution and coverage statistics.
4. Documenting coverage progression, analyzing subtle logical edge cases, and justifying why statement coverage alone is insufficient.

2. Application Architecture and Business Rules

The module under test (marathon_registration.py) provides three core functions managing event registration, wave routing, and result categorization:

1. calculate_entry_fee: Computes registration costs according to distance ('5k': $25, '10k': $40, 'half': $65, 'full': $90). Minors under 12 are rejected outright with ValueError('too young'). Runners under 18 attempting 'half' or 'full' distances are rejected with ValueError('too young for this distance'). Eligible runners qualify for discounts: juniors under 18 receive $5 off

(5k/10k), seniors aged 60+ receive $10 off, and early-bird registrants receive a 20% discount applied after age deductions.

2. assign_wave: Assigns runners to starting waves ('Wave A', 'Wave B', 'Wave C', or 'Wave D') based on qualifying times and bib number fallbacks. 'full' marathons route times < 180 mins to Wave A, < 240 mins to Wave B, and >= 240 mins to Wave C (with bib fallback: < 1000 -> Wave A, < 5000 -> Wave B, else Wave C). 'half' marathons route times < 90 mins to Wave A, < 120 mins to Wave B, else Wave C (bib fallback: < 2000 -> Wave A, < 8000 -> Wave B, else Wave C). '10k' routes times < 45 mins to Wave B, else Wave C. '5k' routes bib < 500 to Wave C, else Wave D.

3. determine_finisher_status: Maps participant outcomes to categorical statuses: 'Disqualified' if disqualified flag is True, 'Did Not Finish' if dnf flag is True, 'Time Limit Exceeded' if chip time exceeds 21600 seconds (6 hours), or 'Finisher'. Disqualification takes strict precedence over DNF.

3. Baseline Coverage Analysis

The starter test suite (test_marathon_registration.py) was executed to establish baseline coverage. It included only three trivial assertions, executing 14 out of 38 statements (37% statement coverage) and leaving 7 out of 28 branches partially covered (32% branch coverage). Critical edge cases, invalid inputs, discounts, bib number fallbacks, and time limit checks remained completely unexercised.

4. Test Suite Implementation and Coverage Progression

To resolve coverage deficiencies, two dedicated test suites were implemented using pytest and coverage:

Phase 1: Statement Coverage (CodeCoverageTest_marathon_registration.py): A test suite containing 15 test cases structured to execute every executable line of marathon_registration.py at least once. This suite reached 100% statement coverage (38/38 statements), successfully exceeding the 90% requirement. However, evaluating it with branch coverage flags revealed only 75% branch coverage due to missed decision outcomes in compound predicates.

Phase 2: Branch Coverage (BranchCoverageTest_marathon_registration.py): A rigorous suite containing 25 test cases specifically targeting every decision branch, control structure, and Boolean short-circuit path. It verifies both True and False evaluations for all 28 decision branches (including race in ('half', 'full') with age < 18 vs age >= 18, and None vs non-None bib/qualifying values), reaching 100% branch coverage (28/28 branches, 0 partial).

Coverage Progression Summary Table

Test Suite	Stmts	Miss	Stmt Cov	Branches	BrPart	Branch Cov	Target Status
Baseline (test_marathon_registration.py)	38	24	37%	28	7	32%	Baseline
Phase 1: CodeCoverageTest_...	38	0	100%	28	7	75%	Exceeded (>= 90%)
Phase 2: BranchCoverageTest_...	38	0	100%	28	0	100%	Exceeded (>= 90%)

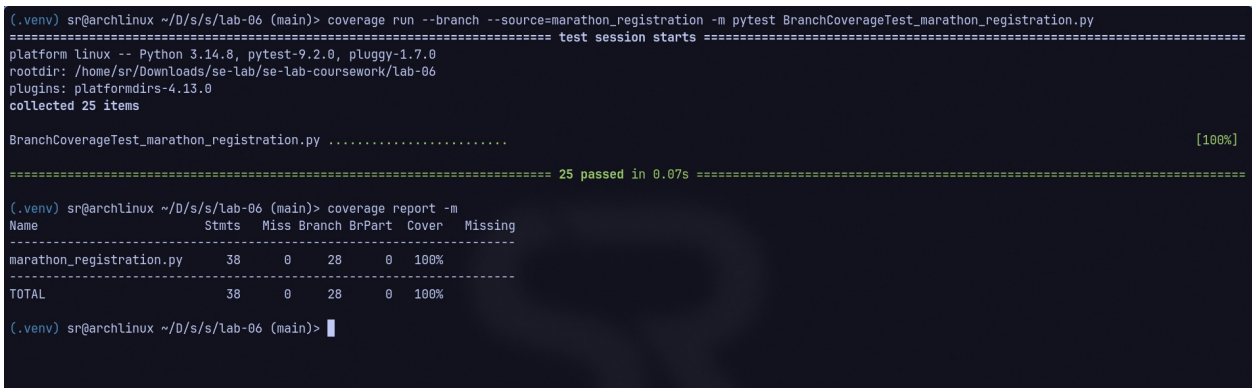
5. Terminal Execution Verification

The following captures show the actual Alacritty terminal output executing both test suites with Python coverage:

Figure 1: Statement Coverage Terminal Execution (100% Statement Coverage)



Figure 2: Branch Coverage Terminal Execution (100% Branch Coverage)



6. In-Depth Comparative Analysis: Statement vs. Branch Coverage

A central finding of this lab is demonstrating that 100% statement coverage is a necessary but insufficient condition for high software reliability:

1. Hidden Paths in Compound Predicates: In marathon_registration.py, the eligibility guard check evaluates if race in ('half', 'full') and age < 18. Under statement coverage, supplying race='half' and age=16 triggers the ValueError, executing the statement. However, statement coverage does not verify what happens when the first clause is True but the second clause is False (an adult registering for a half/full marathon). Without branch coverage, tests may omit verifying that eligible adults properly bypass this exception block.

2. Multi-Tier Fallback Validation: In the wave assignment logic, decisions rely on fallbacks: if qualifying_time_minutes is not None, followed by else: if bib_number is not None and bib_number < 1000. Achieving statement coverage only requires exercising one path to each return value. Branch coverage demands evaluating both None and non-None permutations, guaranteeing that missing data inputs (e.g., bib_number=None) fall through gracefully to the open wave rather than crashing.

3. Boundary Value Completeness: Branch coverage rigorously tests boundary values: age 11 vs 12, age 17 vs 18, age 59 vs 60, qualifying time 179 vs 180, and chip time 21600 (6 hours) vs 21601. This ensures that off-by-one errors in relational operators (< vs <=) are detected.

7. Conclusion

This laboratory demonstrated the systematic progression of software testing from ad-hoc baseline tests (37% statement, 32% branch coverage) to comprehensive statement coverage (100%) and ultimately to complete branch coverage (100% across all 28 branches). The resulting test suite guarantees high structural robustness, exercises all compound logical boundaries, and ensures defect-free operation for the Marathon Registration system.